Introdução a Algoritmos

Aula 6 – Teste de Mesa e Laços Aninhados

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula
Ao final desta aula, você deverá ser capaz de:

1. Construir um teste de mesa tabular para um programa com laços, rastreando o valor de cada variável a cada iteração;
2. Usar o teste de mesa para localizar o instante exato em que um programa com bug diverge do comportamento esperado;
3. Implementar uma busca exaustiva sobre um espaço finito usando laços aninhados, mantendo a melhor solução encontrada;
4. Rastrear um laço de três níveis aninhados identificando quantas vezes o bloco mais interno é executado.

1 Por que simular o programa na mão?

O compilador é uma máquina obediente: ele executa exatamente o que você escreveu, não o que você **quis** escrever. Quando um programa produz um resultado errado, o bug não está no compilador — está na diferença entre a sua intenção e o que o código realmente faz.

A ferramenta para revelar essa diferença é o **teste de mesa**: uma simulação manual e sistemática da execução do programa, passo a passo, rastreando o valor de cada variável a cada instrução. É lento comparado a simplesmente rodar o código, mas é o único método que garante que você **entende** o que está acontecendo — em vez de apenas observar o resultado.

Teste de mesa é útil em dois cenários distintos:

- **Entender** um algoritmo novo, rastreando um código correto para ver como ele produz o resultado esperado;
- **Depurar** um código com bug, rastreando a execução para identificar exatamente onde o comportamento diverge do esperado.

Nesta aula, faremos os dois.

2 O formato do teste de mesa

Um teste de mesa bem feito segue um formato tabular: cada coluna representa uma variável ou condição relevante; cada linha representa um instante da execução (tipicamente, cada iteração de um laço ou cada decisão tomada).

Definição — Teste de Mesa
Um teste de mesa é uma tabela na qual cada linha registra o estado do programa — os valores de todas as variáveis relevantes — imediatamente após a execução de uma instrução ou ao final de cada iteração de um laço.

Para construir um teste de mesa:

1. Identifique as variáveis relevantes e crie uma coluna para cada uma.
2. Adicione uma coluna para a condição do laço (ou do `if`), registrando seu valor booleano a cada avaliação.
3. Execute o programa mentalmente, linha a linha, preenchendo a tabela.
4. Se o valor de uma variável não muda numa linha, repita o valor anterior.

Vamos aplicar esse formato imediatamente.

3 Exemplo 1 — Maior e menor de uma sequência (código correto)

O programa abaixo lê `n` inteiros e ao final exibe o maior e o menor valor lido.

```c
#include <stdio.h>

int main() {
    int n, x, maior, menor, i;

    printf("Quantos números? ");
    scanf("%d", &n);

    scanf("%d", &x);
    maior = x;
    menor = x;

    i = 1;
    while (i < n) {
        scanf("%d", &x);
        if (x > maior) maior = x;
        if (x < menor) menor = x;
        i++;
    }

    printf("Maior: %d\n", maior);
    printf("Menor: %d\n", menor);
    return 0;
}
```

Antes de rastrear, leia o código e tente responder: por que lemos o primeiro valor **antes** do laço? Por que o laço vai de 1 até `n-1` (e não de 0 até `n-1`)?

Teste de Mesa
Entrada: `n = 4`, valores lidos em ordem: `7, 2, 9, 4`.

Após a inicialização (`x = 7`, `maior = 7`, `menor = 7`, `i = 1`):

Iteração	x lido	maior	menor	i	i < n

—	7 (init)	7	7	1	$1 < 4 \longrightarrow$ V
1ª	2	7	2	2	$2 < 4 \longrightarrow$ V
2ª	9	9	2	3	$3 < 4 \longrightarrow$ V
3ª	4	9	2	4	$4 < 4 \longrightarrow$ F

Resultado impresso: `Maior: 9`, `Menor: 2`. ✓

Observe que na 2ª iteração apenas `maior` muda (9 > 7); `menor` permanece 2 porque 4 não é menor que 2. O teste de mesa torna essa seletividade visível.

4 Exemplo 2 — O mesmo problema, com um bug

Um colega escreveu a seguinte versão do mesmo programa:

```c
#include <stdio.h>

int main() {
    int n, x, maior, menor, i;

    printf("Quantos números? ");
    scanf("%d", &n);

    maior = 0;   /* BUG AQUI? */
    menor = 0;   /* BUG AQUI? */

    i = 0;
    while (i < n) {
        scanf("%d", &x);
        if (x > maior) maior = x;
        if (x < menor) menor = x;
        i++;
    }

    printf("Maior: %d\n", maior);
    printf("Menor: %d\n", menor);
    return 0;
}
```

À primeira vista, parece razoável: inicializar `maior` e `menor` com zero. Vamos usar o teste de mesa para verificar.

Teste de Mesa

Entrada: `n = 4`, valores: `7, 2, 9, 4`. Estado inicial: `maior = 0`, `menor = 0`, `i = 0`.

Iteração	x lido	maior	menor	i	i < n
—	—	0	0	0	$0 < 4 \longrightarrow$ V
1ª	7	7	0	1	$1 < 4 \longrightarrow$ V

2ª	2	7	0	2	2 < 4 ⟶ V
3ª	9	9	0	3	3 < 4 ⟶ V
4ª	4	9	0	4	4 < 4 ⟶ F

Resultado impresso: `Maior: 9`, `Menor: 0`. ✗

O bug fica evidente na tabela: `menor` nunca é atualizado porque nenhum dos valores lidos é menor que 0. A condição `x < menor` nunca é verdadeira.

O programa funciona **acidentalmente** para entradas com números negativos onde o menor é de fato negativo — o que torna o bug ainda mais traiçoeiro: ele pode passar despercebido em vários testes e só aparecer numa situação específica.

A correção: inicializar `maior` e `menor` com o **primeiro valor lido**, não com zero (nem com qualquer outra constante). Isso é exatamente o que a versão correta faz. A regra geral é:

Nunca inicialize `maior` ou `menor` com uma constante arbitrária (0, −1, 9999...). Inicialize sempre com o primeiro elemento da sequência. Qualquer constante que você escolher será errada para alguma entrada válida.

5 Conversão decimal para binário

Todo número inteiro positivo pode ser expresso em base 2 (binária) usando apenas os dígitos 0 e 1. O algoritmo de conversão é simples: divida o número por 2 repetidamente, anotando os restos. A sequência de restos, lida de baixo para cima, é a representação binária.

Por exemplo: 13 em binário.

Teste de Mesa

n	n % 2 (resto)	n / 2 (próximo n)
13	1	6
6	0	3
3	1	1
1	1	0

Restos de baixo para cima: **1101**. Verificação: 8 + 4 + 0 + 1 = 13. ✓

O problema é que o algoritmo produz os bits na ordem **inversa** — do menos significativo para o mais significativo. Uma solução simples com o ferramental que temos é armazenar os restos numa variável que os acumula como dígitos decimais, multiplicando por potências de 10:

```
#include <stdio.h>

/*
 * Descrição: Lê um inteiro positivo e imprime sua representação binária.
 * Entrada:   inteiro n >= 1
```

```
 * Saída:     representação binária de n (bits do mais para o menos significativo)
 * Estratégia: acumula os bits como dígitos de um número decimal,
 *             usando potências de 10 como posicionamento.
 */
int main() {
    int n, resto, binario = 0, potencia = 1;

    printf("n = ");
    scanf("%d", &n);

    while (n > 0) {
        resto = n % 2;
        binario = binario + resto * potencia;
        potencia = potencia * 10;
        n = n / 2;
    }

    printf("Binário: %d\n", binario);
    return 0;
}
```

Teste de Mesa

Entrada: `n = 13`. Estado inicial: `binario = 0`, `potencia = 1`.

n	resto	binario	potencia	n > 0	13
1	0 + 1×1 = 1	10	V	6	0
1 + 0×10 = 1	100	V	3	1	1 + 1×100 = 101
1000	V	1	1	101 + 1×1000 = 1101	10000
V	0	—	1101	—	F

Resultado: `1101`. ✓ — os bits acumulam na ordem correta porque cada novo resto é colocado numa posição decimal mais alta que os anteriores.

Atenção

Esta implementação tem um limite prático: `binario` é um `int` e vai transbordar para números grandes. Para converter números maiores que 1023 (10 bits), seria necessário usar strings ou arrays — recursos que veremos em aulas futuras. Por ora, o algoritmo ilustra o raciocínio corretamente.

6 Laços aninhados em três níveis

Na Aula 5 vimos laços aninhados em dois níveis (tabuada). Quando acrescentamos um terceiro nível, o número de iterações cresce rapidamente — e rastrear manualmente se torna mais desafiador. O teste de mesa com múltiplos contadores é a ferramenta para dominar esse raciocínio.

6.1 Revisão: dois níveis — triângulo de asteriscos

```
int n = 4;
for (int i = 1; i <= n; i++) {        /* laço externo: linhas */
```

```
    for (int j = 1; j <= i; j++) {    /* laço interno: asteriscos na linha */
        printf("*");
    }
    printf("\n");
}
```

Teste de Mesa

i	j percorre	Saída da linha	i <= n
1	1..1	*	V
2	1..2	**	V
3	1..3	***	V
4	1..4	****	V
5	—	—	F

Resultado visual:

```
*
**
***
****
```

6.2 Três níveis — cubo de coordenadas

Adicionando um terceiro laço, cada par `(i, j)` do laço externo gera uma linha completa de valores produzidos pelo laço mais interno:

```
int n = 3;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        for (int k = 1; k <= n; k++) {
            printf("(%d,%d,%d) ", i, j, k);
        }
        printf("\n");
    }
    printf("---\n");
}
```

Teste de Mesa

Para `n = 2` (simplificado para caber na página):

i	j	k percorre	Saída
1	1	1,2	`(1,1,1) (1,1,2)`
1	2	1,2	`(1,2,1) (1,2,2)`
2	1	1,2	`(2,1,1) (2,1,2)`
2	2	1,2	`(2,2,1) (2,2,2)`

O laço mais interno executa `n` vezes para cada combinação `(i, j)`. Com `n = 3`, o total de execuções do `printf` interno é $3 \times 3 \times 3 = 27$. Em geral, `k` níveis de laços de `n` iterações cada produzem n^k execuções do bloco mais interno.

6.3 Três níveis com acumulação — soma de todos os produtos

Um exemplo mais significativo: calcular a soma de todos os produtos `i * j * k` para `i`, `j`, `k` variando de 1 a `n`:

```
#include <stdio.h>

int main() {
    int n, soma = 0;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            for (int k = 1; k <= n; k++) {
                soma += i * j * k;
            }
        }
    }

    printf("Soma: %d\n", soma);
    return 0;
}
```

Para `n = 2`, o teste de mesa das iterações do laço mais interno:

Teste de Mesa

i	j	k	i*j*k	soma
1	1	1	1	1
1	1	2	2	3
1	2	1	2	5
1	2	2	4	9
2	1	1	2	11
2	1	2	4	15
2	2	1	4	19
2	2	2	8	27

Resultado: `soma = 27`. Verificação algébrica: $(1+2)^3 = 27$ — coincidência? Não: a soma de todos os produtos $i \cdot j \cdot k$ para $i, j, k \in \{1, \ldots, n\}$ é igual a $\left(\sum_{i=1}^{n} i\right)^3$. Para $n = 2$: $(1+2)^3 = 27$. ✓

7 Busca exaustiva: o problema do troco mínimo

Problema: dado um valor de troco `V` (em centavos) e um conjunto de denominações de moedas disponíveis, qual é o menor número de moedas necessário para compor exatamente o valor `V`?

Exemplo: `V = 30` centavos, moedas disponíveis: 1¢, 5¢, 10¢, 25¢.
- Uma solução: 25¢ + 5¢ = 2 moedas. ✓
- Outra solução: 10¢ + 10¢ + 10¢ = 3 moedas. (pior)
- Outra: 1¢ × 30 = 30 moedas. (muito pior)

A estratégia **gulosa** — sempre pegar a moeda de maior valor que caiba — funciona para o sistema monetário brasileiro e americano, mas **falha** em outros sistemas. Considere moedas de 1¢, 3¢ e 4¢, com `V = 6`:
- Estratégia gulosa: 4¢ + 1¢ + 1¢ = 3 moedas.
- Solução ótima: 3¢ + 3¢ = 2 moedas.

A abordagem garantida é a **busca exaustiva** (força bruta): enumerar todas as combinações possíveis e guardar a de menor custo.

7.1 Implementação: moedas de 1¢, 5¢ e 10¢

Para simplificar, vamos trabalhar com três denominações fixas e um troco máximo razoável. Chamamos de `a`, `b`, `c` o número de moedas de 10¢, 5¢ e 1¢ respectivamente, e buscamos minimizar `a + b + c` sujeito a `10*a + 5*b + 1*c == V`.

```c
#include <stdio.h>

/*
 * Descrição: Encontra o número mínimo de moedas (10¢, 5¢, 1¢)
 *            para compor exatamente o valor V (em centavos).
 * Entrada:   V inteiro, 1 <= V <= 100
 * Saída:     quantidade de cada moeda e o total mínimo
 */
int main() {
    int V;
    printf("Troco (centavos): ");
    scanf("%d", &V);

    int melhor = V + 1;   /* pior caso: V moedas de 1¢ */
    int a_melhor = 0, b_melhor = 0, c_melhor = 0;

    /* testa todas as combinações possíveis */
    for (int a = 0; a <= V / 10; a++) {
        for (int b = 0; b <= (V - 10*a) / 5; b++) {
            int c = V - 10*a - 5*b;   /* c é determinado por a e b */
            int total = a + b + c;
            if (total < melhor) {
                melhor    = total;
                a_melhor  = a;
                b_melhor  = b;
                c_melhor  = c;
            }
        }
    }
```

```
    printf("Mínimo: %d moeda(s)\n", melhor);
    printf("  10¢: %d\n   5¢: %d\n   1¢: %d\n",
           a_melhor, b_melhor, c_melhor);
    return 0;
}
```

Observe três decisões de projeto importantes neste programa:

1. **`c` não precisa de um laço próprio.** Dados `a` e `b`, o valor de `c` é determinado: `c = V - 10*a - 5*b`. Isso reduz o espaço de busca de cúbico para quadrático — sem perder nenhuma solução.
2. **`melhor` é inicializado com `V + 1`.** Esse é o pior caso possível (usar V moedas de 1¢). Qualquer solução real será melhor do que isso, garantindo que a primeira combinação válida encontrada já seja registrada.
3. **O laço interno para em `(V - 10*a) / 5`**, não em `V`. Isso evita testar combinações cujo custo em moedas de 5¢ já excede o troco restante.

Teste de Mesa

Entrada: `V = 11`. Limite para `a`: 0..1. Vamos rastrear todas as iterações:

a	b	c	total	melhor
0	0	11	11	11
0	1	6	7	7
0	2	1	3	3
1	0	1	2	2
1	—	—	—	2 ⟵ `(V-10)/5 = 0`, laço b não executa mais

Resultado: 1 moeda de 10¢ + 0 de 5¢ + 1 de 1¢ = 2 moedas. ✓

A linha `(a=0, b=2, c=1)` é interessante: usa 3 moedas e é a estratégia gulosa com moedas de 5¢. A busca exaustiva encontra a solução de 2 moedas porque não descarta nenhuma possibilidade prematuramente.

7.2 A ideia geral da busca exaustiva

O troco mínimo é uma instância de uma família ampla de problemas: **encontrar a melhor solução dentre todas as possíveis**, quando o espaço de possibilidades é finito e enumerável.

A estrutura é sempre a mesma:

```
melhor = PIOR_CASO_POSSIVEL;

for (/* todas as possibilidades */) {
    if (/* possibilidade é válida */ && /* é melhor que a atual */) {
        melhor = /* atualiza */;
    }
}
```

Busca exaustiva é garantidamente correta — nunca perde a solução ótima — mas pode ser lenta quando o espaço de busca é grande. Para o troco com três denominações e `V ≤ 100`, o número de iterações

é no máximo algumas centenas: perfeitamente viável. Para problemas maiores, existem técnicas mais eficientes (programação dinâmica, algoritmos gulosos com prova de correção), que aparecem em disciplinas posteriores.

8 Síntese: o que o teste de mesa nos ensinou

Ao longo desta aula, o teste de mesa cumpriu papéis diferentes em cada exemplo:

- **Maior e menor (correto):** mostrou por que a inicialização com o primeiro elemento é necessária — sem isso, o invariante do laço não vale.
- **Maior e menor (com bug):** localizou o erro na inicialização com zero, explicando **por que** o programa falha apenas para entradas específicas.
- **Conversão binária:** revelou como os bits se acumulam na ordem correta, tornando visível um aspecto do algoritmo que é difícil de ver só lendo o código.
- **Busca exaustiva:** confirmou que todas as combinações relevantes são testadas e que a atualização de `melhor` ocorre nos momentos certos.

Em todos os casos, o teste de mesa transforma o programa de uma sequência de símbolos numa **execução compreendida**. Essa compreensão é o que distingue um programador que depura sistematicamente de um que tenta correções ao acaso.

9 Exercícios

1. Faça o teste de mesa do programa de conversão binária para `n = 10`. Verifique o resultado convertendo 10 para binário manualmente (1010_2).

2. O programa abaixo deveria calcular a soma dos quadrados de 1 a `n`, mas produz resultados errados. Faça o teste de mesa para `n = 3` e identifique o bug. Corrija-o.

```
int soma = 0, i = 0;
while (i <= n) {
    soma += i * i;
    i++;
}
printf("%d\n", soma);   /* esperado para n=3: 1+4+9 = 14 */
```

3. Escreva um programa que leia `n` números reais e calcule a média, o maior e o menor valor. Inicialize corretamente. Faça o teste de mesa para a entrada `n = 5`, valores: `3.0, 7.5, 1.2, 9.0, 4.8`.

4. Estenda o programa de busca exaustiva do troco para incluir também moedas de 25¢. Quantos laços são necessários? Você consegue eliminar algum deles pelo mesmo raciocínio que eliminou o laço de `c`?

5. *(Desafio)* Usando busca exaustiva, escreva um programa que encontre todos os pares de inteiros positivos `(a, b)` com `a ≤ b ≤ 100` tais que `a² + b²` seja um quadrado perfeito (ou seja, `a`, `b`, `c` formam um triplo pitagórico com `c ≤ 100`). Quantos triplos existem?